

Adquisición de datos de corriente y potencia AC con microcontrolador

Ingeniería Mecánica y Electromecánica

Diego Alejandro Cruz Torrez
dacruz9@umsa.bo
Camacho Salgado Jorge Fernando
contacto@correo.com
Ing. Vidher Bustillos

Resumen

La medición de corriente alterna en sistemas eléctricos es fundamental para el análisis del consumo energético y la caracterización de cargas. En este trabajo se presenta la implementación de un sistema de adquisición de datos para medir la corriente consumida por un foco incandescente utilizando un sensor de corriente no invasivo SCT-013. Debido a la baja amplitud de la señal entregada por el sensor, se diseñó un circuito acondicionador basado en un amplificador operacional en configuración no inversora con una ganancia de 13, además de emplear múltiples vueltas del conductor alrededor del sensor para incrementar la sensibilidad de medición. La señal acondicionada es adquirida por un microcontrolador (Arduino Nano), el cual digitaliza y transmite los datos para su análisis. Los resultados permiten observar la forma de onda de la corriente alterna y estimar su valor eficaz. Este sistema constituye una solución práctica para la medición segura de corriente en sistemas de baja potencia sin necesidad de contacto eléctrico directo.

Palabras clave: corriente alterna, SCT-013, adquisición de datos, amplificador operacional, sensor de corriente.

AC current data acquisition using SCT-013 sensor and microcontroller

Abstract

The measurement of alternating current is essential for energy consumption analysis and load characterization. This work presents the implementation of a data acquisition system to measure the current consumed by an incandescent lamp using a non-invasive SCT-013 current sensor. Due to the low amplitude of the sensor output signal, a signal conditioning circuit based on a non-inverting operational amplifier with a gain of 13 was designed. Additionally, multiple turns of the conductor around the sensor were used to increase measurement sensitivity. The conditioned signal is acquired by a microcontroller, which digitizes and transmits the data for analysis. The results allow observing the AC current waveform and estimating its RMS value. This system represents a practical solution for safe current measurement in low-power systems without direct electrical contact.

Keywords: alternating current, SCT-013, data acquisition, operational amplifier, current sensor.

Introducción o Antecedentes

La medición de corriente alterna es una tarea clave en el análisis de sistemas eléctricos, especialmente en aplicaciones relacionadas con eficiencia energética, monitoreo de consumo y diagnóstico de cargas. A diferencia de la medición de corriente continua, la corriente alterna requiere técnicas que permitan capturar señales variables en el tiempo y procesarlas adecuadamente para obtener parámetros como el valor eficaz (RMS).

Los sensores de corriente tipo transformador de corriente (CT), como el SCT-013, permiten medir corriente de manera no invasiva mediante el principio de inducción electromagnética. Estos sensores generan una señal proporcional a la corriente que circula por el conductor primario, proporcionando aislamiento eléctrico y mayor seguridad en la medición.

Sin embargo, la señal de salida de estos sensores suele ser de baja amplitud, por lo que es necesario emplear circuitos de acondicionamiento que permitan amplificar y adaptar la señal al rango de entrada del microcontrolador. En este contexto, el uso de amplificadores operacionales en configuración no inversora permite aumentar la amplitud de la señal sin alterar su forma.

El objetivo de este trabajo es implementar un sistema de adquisición de datos capaz de medir la corriente alterna consumida por un foco incandescente utilizando un sensor SCT-013, un circuito amplificador y un microcontrolador.

Desarrollo de la Investigación

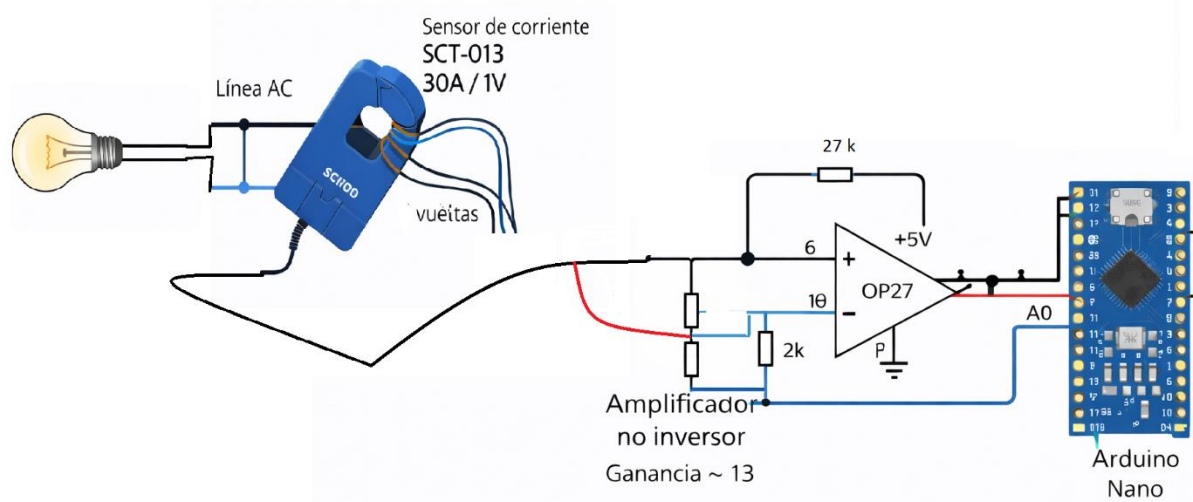
Solución propuesta

La solución consiste en medir la corriente alterna mediante un sensor SCT-013, cuya salida es proporcional a la corriente que circula por el conductor. Para mejorar la sensibilidad del sistema, se realizaron 6 vueltas del cable alrededor del sensor, lo que incrementa la corriente efectiva detectada.

La señal obtenida es acondicionada mediante un amplificador operacional en configuración no inversora con una ganancia de 13, elevando la amplitud de la señal a niveles adecuados para su adquisición por el microcontrolador.

Finalmente, el microcontrolador digitaliza la señal y la envía a un sistema de visualización para su análisis.

Figura 1: Diagrama de conexonado



. Fuente: Elaboración propia

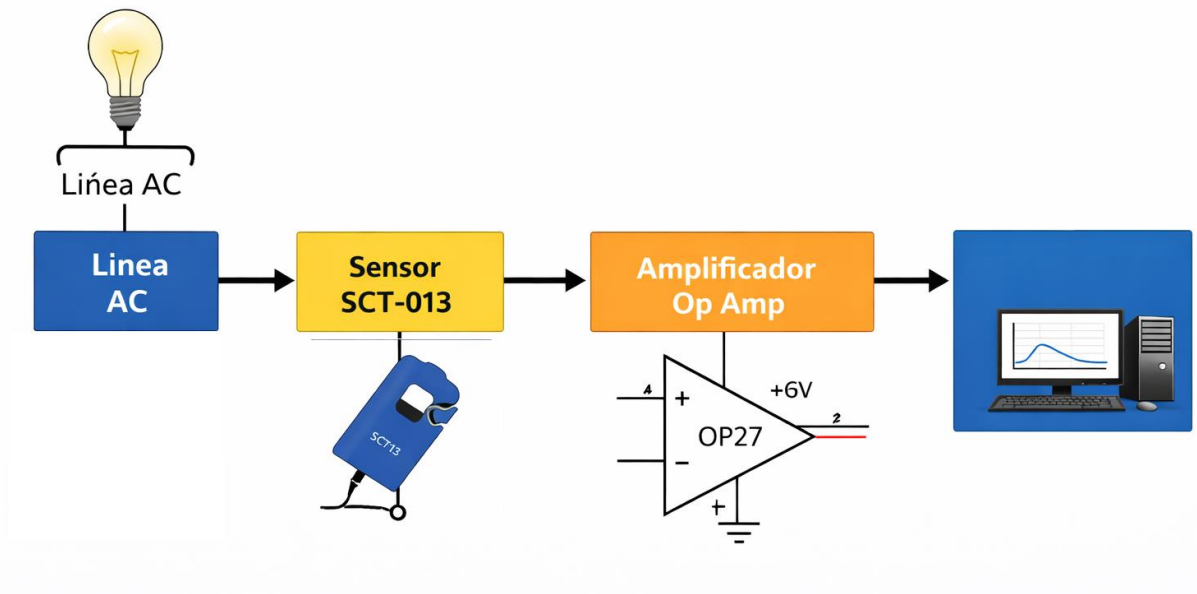
Requerimientos materiales y equipos

Para la implementación del sistema se utilizaron los siguientes componentes:

- Microcontrolador
- Sensor de corriente SCT-013 (30A / 1V)
- Foco incandescente
- Amplificador operacional
- Resistencias para configuración de ganancia
- Protoboard
- Cables de conexión
- Computador
- Software de programación

Diagrama en bloques

Figura 2: Diagrama en bloques



Fuente: Elaboración propia

Codificación de los programas

Codigo 1: Codigo en Arduino

```
int sensorPin = A0;    // Entrada analogica del Arduino
int sensorValue = 0;   // variable que almacena el valor del sensor
void setup() {
  Serial.begin(115200);
}
void loop() {
  // lectura del valor y escritura por datos seriales
  sensorValue = analogRead(sensorPin);
  Serial.println(sensorValue);
  delay(100);
}
```

Codigo 2: Codigo en Python

```

import serial
import tkinter as tk
from tkinter import Frame, Label
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
import matplotlib.pyplot as plt
import matplotlib.animation as animation
import csv
import time
import math
from collections import deque
# VARIABLES ESTATICAS
VREF_ADC = 5.0
ADC_MAX = 1023.0
VOLTAJE_RED = 230.0
FS = 1000 # 1 muestra cada 1 ms
INTERVALO_GRAFICO_MS = 20
MUESTRAS_GRAFICO = 400
MUESTRAS_RMS = 1000
MUESTRAS_OFFSET = 500
VENTANA_FILTRO = 5
OFFSET_TEORICO_V = 2.32
# Corriente_real(A) =K_CAL, K_CAL=1/15*1/6*30/1
K_CAL1 = 0.267
K_CAL = 0.267
#Conexion al arduino
arduino = serial.Serial('COM3', baudrate=115200, timeout=1)
#Archivo CSV
archivo = open("medicion.csv", "w", newline="")
writer = csv.writer(archivo)
writer.writerow(["Tiempo (s)", "Corriente RMS (mA)", "Potencia (W)", "Energia (kWh)"])
# INTERFAZ GRAFICA
root = tk.Tk()
root.title("Monitor de Corriente AC - SCT013")
root.geometry("1100x700")
root.configure(bg="#f2f4f7")
titulo = Label(
    root,
    text="Monitor",
    font=("Arial", 20, "bold"),
    bg="#f2f4f7",
    fg="#1f2937",
    pady=10
)
titulo.pack()
# ----- Panel superior -----
panel = Frame(root, bg="#f2f4f7")
panel.pack(pady=10)
def crear_tarjeta(parent, titulo_texto, valor_inicial):
    tarjeta = Frame(parent, bg="white", bd=1, relief="solid", padx=20, pady=15)
    titulo = Label(
        tarjeta,
        text=titulo_texto,
        font=("Arial", 12, "bold"),
        bg="white",
        fg="#374151"

```

```

    )
    titulo.pack()
    valor = Label(
        tarjeta,
        text=valor_inicial,
        font=("Arial", 18, "bold"),
        bg="white",
        fg="#111827"
    )
    valor.pack(pady=(8, 0))
    return tarjeta, valor
tarjeta1, label_irms = crear_tarjeta(panel, "Corriente RMS", "0.000 A")
tarjeta1.grid(row=0, column=0, padx=15)
tarjeta2, label_pot = crear_tarjeta(panel, "Potencia", "0.0 W")
tarjeta2.grid(row=0, column=1, padx=15)
tarjeta3, label_energia = crear_tarjeta(panel, "Energía", "0.000000 kWh")
tarjeta3.grid(row=0, column=2, padx=15)
fig, ax = plt.subplots(figsize=(11, 4.8))
fig.patch.set_facecolor("#f2f4f7")
ax.set_facecolor("white")
linea, = ax.plot([], [], lw=2)
ax.set_title("Señal sinusoidal de corriente", fontsize=14)
ax.set_xlabel("Tiempo (ms)")
ax.set_ylabel("Corriente instantánea (A)")
ax.grid(True, alpha=0.3)
canvas = FigureCanvasTkAgg(fig, master=root)
canvas.get_tk_widget().pack(pady=10)
# VARIABLES
inicio = time.time()
ultimo_tiempo = inicio
ultimo_panel = inicio
energia_kwh = 0.0
contador = 0
x_data = deque(maxlen=MUESTRAS_GRAFICO)
y_data = deque(maxlen=MUESTRAS_GRAFICO)
buffer_corriente = deque(maxlen=MUESTRAS_RMS)
buffer_offset = deque(maxlen=MUESTRAS_OFFSET)
# Para filtro
buffer_filtro = deque(maxlen=VENTANA_FILTRO)
# FUNCIONES
def adc_a_voltaje(adc):
    return (adc * VREF_ADC) / ADC_MAX
def calcular_rms(muestras):
    if len(muestras) == 0:
        return 0.0
    return math.sqrt(sum(i**2 for i in muestras) / len(muestras))
def init():
    ax.set_xlim(0, MUESTRAS_GRAFICO)
    ax.set_ylim(-1, 1)
    linea.set_data([], [])
    return linea,
def actualizar(frame):
    global contador, energia_kwh, ultimo_tiempo, ultimo_panel
    lectura = arduino.readline().decode(errors='ignore').strip()
    try:

```

```

        adc = int(lectura)
    except ValueError:
        return linea,
    tiempo_actual = time.time()
    t = tiempo_actual - inicio
    dt_horas = (tiempo_actual - ultimo_tiempo) / 3600.0
    ultimo_tiempo = tiempo_actual
    # 1) ADC -> voltaje
    v_a0 = adc_a_voltaje(adc)
    # 2) Quitar offset
    v_ac = v_a0 - OFFSET_TEORICO_V
    # 3) Convertir a corriente real
    i_inst = v_ac * K_CAL
    igraf = i_inst
    # 4) Guardar para grafico y RMS
    x_data.append(contador)
    y_data.append(igraf)
    buffer_corriente.append(i_inst)
    contador += 1
    # 5) Calcular RMS
    i_rms = calcular_rms(buffer_corriente)
    # 6) Potencia
    potencia = VOLTAJE_RED * i_rms
    # 7) Energia
    energia_kwh += (potencia * dt_horas) / 1000.0
    # 8) Actualizar panel cada 200 ms
    if tiempo_actual - ultimo_panel >= 0.5:
        label_irms.config(text=f"{i_rms:.3f} A")
        label_pot.config(text=f"{potencia:.1f} W")
        label_energia.config(text=f"{energia_kwh:.6f} kWh")
        ultimo_panel = tiempo_actual
    # 9) Guardar CSV
    writer.writerow([
        round(t, 4),
        round(i_rms, 5),
        round(potencia, 3),
        round(energia_kwh, 8)
    ])
    archivo.flush()
    # 10) Actualizar grafico
    linea.set_data(list(x_data), list(y_data))
    if len(x_data) > 1:
        ax.set_xlim(min(x_data), max(x_data))
    if len(y_data) > 10:
        ymin = min(y_data)
        ymax = max(y_data)
        margen = max(0.05, (ymax - ymin) * 0.2)
        ax.set_ylim(ymin - margen, ymax + margen)
    return linea,

# CIERRE
ani = animation.FuncAnimation(
    fig,
    actualizar,
    init_func=init,
    interval=INTERVALO_GRAFICO_MS,

```

```

    blit=False,
    cache_frame_data=False
)
def cerrar():
    archivo.close()
    arduino.close()
    root.destroy()

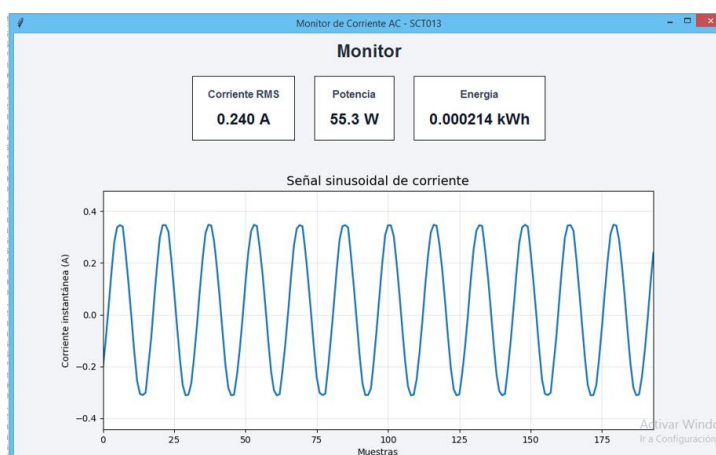
root.protocol("WM_DELETE_WINDOW", cerrar)
root.mainloop()

```

Resultados Obtenidos

Tras realizar el armado del circuito detallado anteriormente se procede a cargar el Arduino con el código descrito en la anterior sección, donde se obtuvo la siguiente gráfica:

Figura 3: Grafica en Python



Fuente: Elaboración propia

Tabla 1: Datos almacenados en CSV

Tiempo (s)	Corriente RMS (mA)	Potencia (W)	Energia (kWh)
20.1761	0.10114	23.262	0.00012573
20.2512	0.10096	23.221	0.00012621
20.3142	0.10078	23.18	0.00012662
20.3783	0.10061	23.14	0.00012703
20.4634	0.10043	23.1	0.00012757
20.5264	0.10317	23.729	0.00012799
20.5904	0.10324	23.745	0.00012841
20.6545	0.10306	23.703	0.00012883
20.7185	0.10288	23.662	0.00012926
20.7826	0.1027	23.621	0.00012968
20.8456	0.10252	23.58	0.00013009
20.9097	0.10234	23.539	0.00013051
20.9747	0.10217	23.499	0.00013093

21.0367	0.10389	23.896	0.00013134
21.1008	0.10384	23.883	0.00013177
21.1638	0.10367	23.843	0.00013219
21.2269	0.10349	23.803	0.0001326
21.2899	0.10332	23.763	0.00013302
21.352	0.10315	23.724	0.00013343
21.416	0.10298	23.685	0.00013385
21.4791	0.10281	23.646	0.00013426
21.5431	0.1039	23.897	0.00013469
21.6062	0.10385	23.886	0.00013511
21.6702	0.10368	23.846	0.00013553
21.7323	0.10351	23.807	0.00013594
21.7953	0.10334	23.768	0.00013636
21.8593	0.10317	23.729	0.00013678
21.9224	0.103	23.69	0.00013719
21.9864	0.10283	23.652	0.00013762
22.0515	0.10324	23.746	0.00013804
22.1165	0.10313	23.719	0.00013847
22.1816	0.10296	23.682	0.0001389
22.2466	0.1028	23.644	0.00013933
22.3126	0.10264	23.607	0.00013976
22.3777	0.10248	23.569	0.00014019
22.4437	0.10231	23.532	0.00014062
22.5088	0.10215	23.496	0.00014104
22.5748	0.10281	23.645	0.00014148
22.6409	0.10273	23.627	0.00014191
22.7049	0.10257	23.59	0.00014233
22.77	0.10241	23.553	0.00014276
22.835	0.10225	23.516	0.00014318
22.9001	0.10209	23.48	0.0001436
22.9651	0.10193	23.444	0.00014403
23.0312	0.10177	23.407	0.00014446
23.0982	0.10234	23.538	0.0001449
23.1723	0.10221	23.508	0.00014538
23.2503	0.10205	23.472	0.00014589
23.3164	0.1019	23.437	0.00014632
23.3814	0.10175	23.402	0.00014674
23.4485	0.1016	23.367	0.00014718
23.5135	0.10144	23.332	0.0001476
23.5796	0.10129	23.297	0.00014803
23.6526	0.10205	23.471	0.0001485

23.7176	0.10194	23.446	0.00014893
23.7827	0.10178	23.411	0.00014935
23.8477	0.10163	23.376	0.00014977
23.9128	0.10148	23.341	0.00015019
23.9778	0.10133	23.306	0.00015061

Fuente: Elaboración propia

Discusión y conclusiones

- El sistema implementado permitió medir de manera efectiva la corriente alterna consumida por un foco incandescente utilizando un sensor no invasivo SCT-013. La combinación del sensor con un circuito amplificador resultó fundamental para adaptar la señal al rango de operación del microcontrolador.
- Se comprobó que el uso de múltiples vueltas del conductor alrededor del sensor incrementa la sensibilidad del sistema, lo cual es especialmente útil en aplicaciones de baja corriente.
- Asimismo, el acondicionamiento de la señal mediante un amplificador no inversor permitió mejorar la calidad de la señal adquirida, reduciendo el impacto del ruido y facilitando su procesamiento digital.
- Se obtuvieron resultados satisfactorios al utilizar focos de diversas potencias (100W, 60W, y 20W), así como focos LED

En conclusión, el sistema desarrollado constituye una solución segura y eficiente para la medición de corriente alterna, siendo especialmente útil en aplicaciones de monitoreo energético y sistemas embebidos.

Referencias

ON Semiconductor. (2016). *LM358 Low Power Dual Operational Amplifier* (Datasheet). ON Semiconductor. <https://www.onsemi.com>

Arduino. (2019). Arduino Nano (Technical specifications and documentation). Arduino. <https://www.arduino.cc>

YHDC. (2018). SCT-013 Current Transformer Sensor (Datasheet). YHDC.

Python Software Foundation. (2023). Python documentation. <https://docs.python.org>

Python Crash Course. (2019). Python Crash Course: A Hands-On, Project-Based Introduction to Programming (2nd ed.). No Starch Press.

ANEXOS



Circuito Físico Implementado. Fuente: Elaboración propia